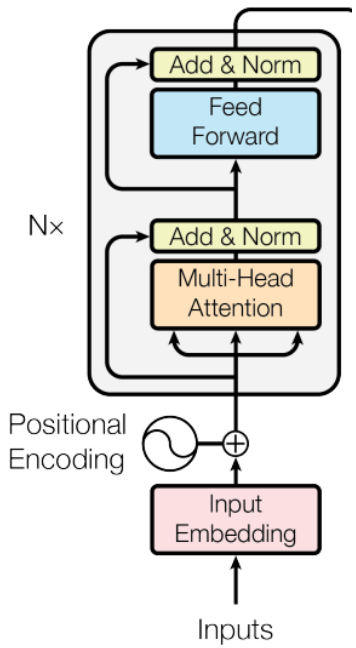
Deep Learning for NLP

Transformer
The Encoder

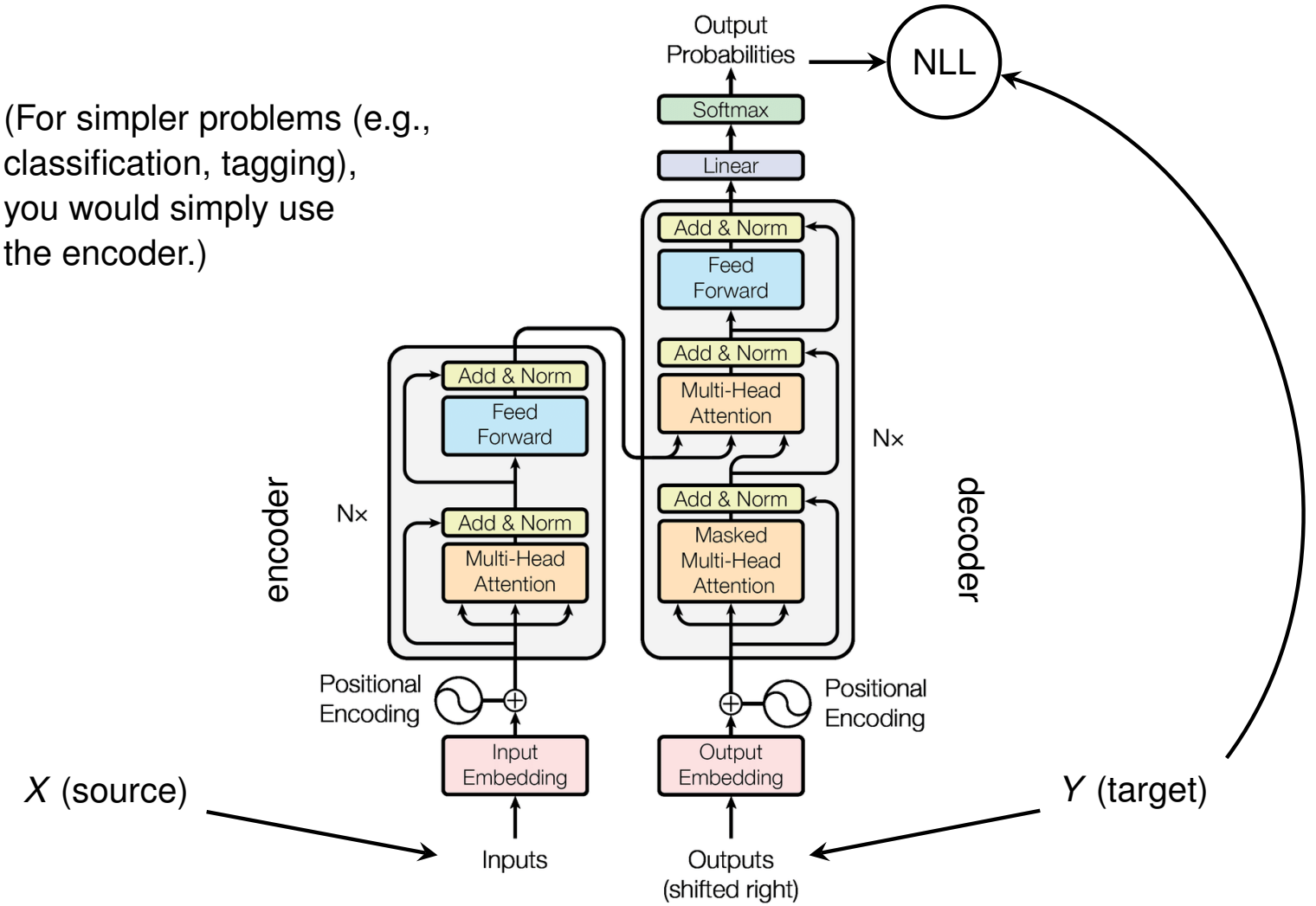


Learning goals

- Understand Self-Attention and the role of position embeddings
- Understand subtleties of parallelized multi-head attention

THE TRANSFORMER ARCHITECTURE I

(For simpler problems (e.g., classification, tagging), you would simply use the encoder.)



▸ Source: Vaswani et al., 2017

ATTENTION IN THE TRANSFORMER I

- We can use attention on many different “things”, including:
  - The pixels of images
  - The nodes of knowledge graphs
  - The words of a vocabulary
- Here, we focus on scenarios where the query, key and value vectors represent tokens (e.g., words, characters, etc.) in sequences (e.g., sentences, paragraphs, etc.).

ATTENTION IN THE TRANSFORMER I

Cross-Attention:

- Let $X = (x_1 \ldots x_{J_x})$, $Y = (y_1 \ldots y_{J_y})$ be two sequences (e.g., source and target in a sequence-to-sequence problem)
- The query vectors represent tokens in Y and the key/value vectors represent tokens in X ("Y attends to X")

Self-Attention:

- There is only one sequence $X = (x_1 \ldots x_J)$
- The query, key and value vectors represent tokens in X ("X attends to itself")

CROSS-ATTENTION (1) I

- We start by describing cross-attention, self-attention can easily be derived by assuming $\vec{X} = \vec{Y}$.
- Let $\vec{X} \in \mathbb{R}^{J_x \times d_x}$, $\vec{Y} \in \mathbb{R}^{J_y \times d_y}$ be representations of X, Y (e.g., stacked word embeddings, or the outputs of a previous layer)
- Let $\theta = \{\vec{W}^{(q)} \in \mathbb{R}^{d_y \times d_q}, \vec{W}^{(k)} \in \mathbb{R}^{d_x \times d_k}, \vec{W}^{(v)} \in \mathbb{R}^{d_x \times d_v}\}$ be trainable weight matrices
- We transform $\vec{Y}$ into a matrix of query vectors:

$$\vec{Q} = \vec{Y}\vec{W}^{(q)}$$

- We transform $\vec{X}$ into matrices of key and value vectors:

$$\vec{K} = \vec{X}\vec{W}^{(k)}; \qquad \vec{V} = \vec{X}\vec{W}^{(v)}$$

CROSS-ATTENTION (2) I

- To calculate the *e* scores, Vaswani et al. use a parameter-less scaled dot product instead of Bahdanau's complicated FFN
- Step 1 of the basic recipe: *Scores*

$$e_{j,j'} = a(\vec{q}_j, \vec{k}_{j'}) = \frac{\vec{q}_j^T \vec{k}_{j'}}{\sqrt{d_k}}$$

- Note: This requires that $d_q = d_k$

CROSS-ATTENTION (3) I

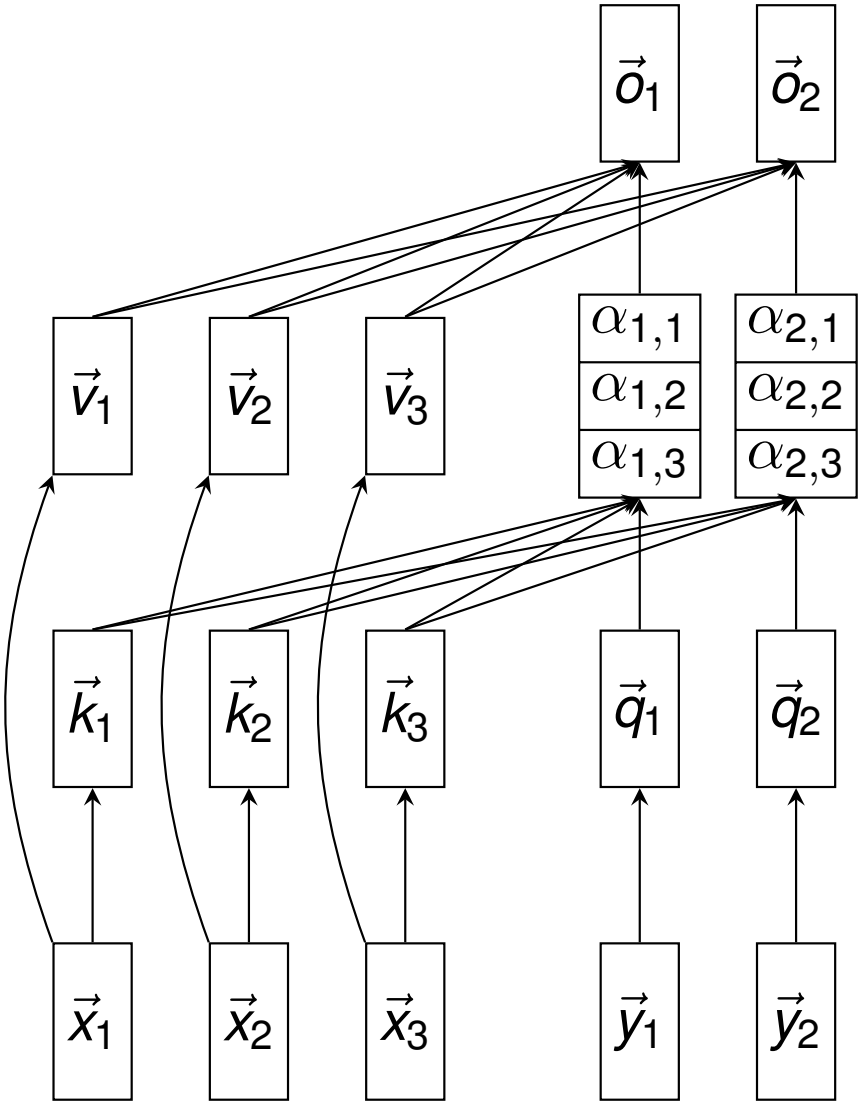
- Attention weights and outputs are defined like before
- Step 2 of the basic recipe: *Weights*

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J_x} \exp(e_{j,j''})}$$

- Step 3 of the basic recipe: *Outputs*

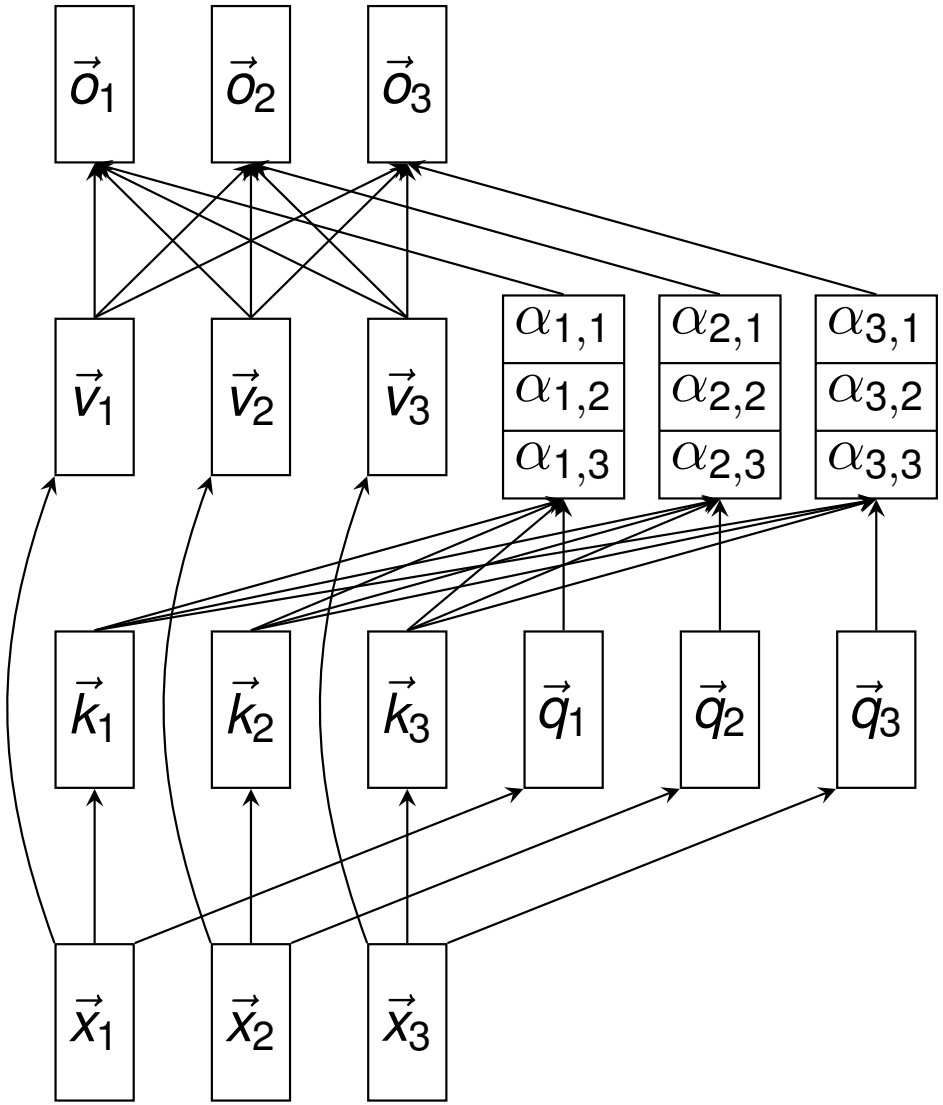
$$\vec{o}_j = \sum_{j'=1}^{J_x} \alpha_{j,j'} \vec{v}_{j'}$$

CROSS-ATTENTION (4) I



Cross-attention

SELF-ATTENTION I



Self-attention

SELF-ATTENTION FORMALIZED I

- Let $\vec{X} \in \mathbb{R}^{J_x \times d_x}$ be a representation of X (e.g., stacked word embeddings, or the outputs of a previous layer)
- Let $\theta = \{\vec{W}^{(q)} \in \mathbb{R}^{d_x \times d_q}, \vec{W}^{(k)} \in \mathbb{R}^{d_x \times d_k}, \vec{W}^{(v)} \in \mathbb{R}^{d_x \times d_v}\}$ be trainable weight matrices
- We transform $\vec{X}$ into a matrix of query vectors:

$$\vec{Q} = \vec{X}\vec{W}^{(q)}$$

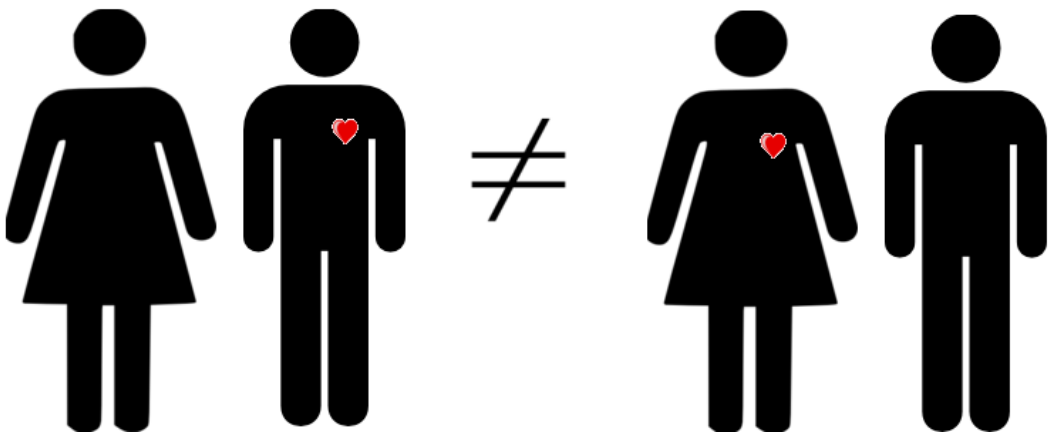
- And we transform $\vec{X}$ into matrices of key and value vectors:

$$\vec{K} = \vec{X}\vec{W}^{(k)}; \qquad \vec{V} = \vec{X}\vec{W}^{(v)}$$

- Execute Steps 1 – 3 of the basic recipe

CAN SELF-ATTENTION MODEL WORD ORDER? I

- Hypothetical model:
  - simple word embedding lookup layer
  - self-attention layer
- Let $X^{(1)}$, $X^{(2)}$ be two sentences of the same length J, which contain the same words, but in a different order
- Example: "john loves mary" vs. "mary loves john"



CAN SELF-ATTENTION MODEL WORD ORDER? I

- Definition of $\vec{o}_j$:

$$\vec{o}_j = \sum_{j'=1}^{J} \alpha_{j,j'} \vec{v}_{j'}$$

- Since addition is commutative, and the permutation is bijective, it is sufficient to show that:

$$\forall_{j \in \{1,\ldots,J\}, j' \in \{1,\ldots,J\}} \alpha^{(1)}_{j,j'} \vec{v}^{(1)}_{j'} = \alpha^{(2)}_{g_j,g_{j'}} \vec{v}^{(2)}_{g_{j'}}$$

CAN SELF-ATTENTION MODEL WORD ORDER? I

- Step 1: Let's show that $\forall_j \vec{v}_j^{(1)} = \vec{v}_{g_j}^{(2)}$
- Definition of $\vec{v}_j$:

$$\vec{v}_j = \vec{W}^{(v)T} \vec{x}_j$$

- Then:

$$\vec{x}_j^{(1)} = \vec{x}_{g_j}^{(2)} \implies \vec{W}^{(v)T} \vec{x}_j^{(1)} = \vec{W}^{(v)T} \vec{x}_{g_j}^{(2)} \implies \vec{v}_j^{(1)} = \vec{v}_{g_j}^{(2)}$$

CAN SELF-ATTENTION MODEL WORD ORDER? I

- Step 2: Let's show that $\forall_{j \in \{1,\ldots,J\}, j' \in \{1,\ldots,J\}} \alpha^{(1)}_{j,j'} = \alpha^{(2)}_{g_j, g_{j'}}$
- Definition of $\alpha_{j,j'}$:

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J} \exp(e_{j,j''})}$$

- Since the sum in the denominator is commutative, and the permutation is bijective, it is sufficient to show that

$$\forall_{j \in \{1,\ldots,J\}, j' \in \{1,\ldots,J\}} e^{(1)}_{j,j'} = e^{(2)}_{g_j, g_{j'}}$$

CAN SELF-ATTENTION MODEL WORD ORDER? I

- Definition of $e_{j,j'}$:

$$e_{j,j'} = \frac{1}{\sqrt{d_k}} \vec{q}_j^T \vec{k}_{j'} = \frac{1}{\sqrt{d_k}} (\vec{W}^{(q)T} \vec{x}_j)^T (\vec{W}^{(k)T} \vec{x}_{j'})$$

- Then:

$$\vec{x}_j^{(1)} = \vec{x}_{g_j}^{(2)} \wedge \vec{x}_{j'}^{(1)} = \vec{x}_{g_{j'}}^{(2)}$$

$$\implies \vec{W}^{(q)T} \vec{x}_j^{(1)} = \vec{W}^{(q)T} \vec{x}_{g_j}^{(2)} \wedge \vec{W}^{(k)T} \vec{x}_{j'}^{(1)} = \vec{W}^{(k)T} \vec{x}_{g_{j'}}^{(2)}$$

$$\implies \vec{q}_j^{(1)} = \vec{q}_{g_j}^{(2)} \wedge \vec{k}_{j'}^{(1)} = \vec{k}_{g_{j'}}^{(2)}$$

$$\implies \vec{q}_j^{(1)T} \vec{k}_{j'}^{(1)} = \vec{q}_{g_j}^{(2)T} \vec{k}_{g_{j'}}^{(2)}$$

$$\implies \frac{1}{\sqrt{d_k}} \vec{q}_j^{(1)T} \vec{k}_{j'}^{(1)} = \frac{1}{\sqrt{d_k}} \vec{q}_{g_j}^{(2)T} \vec{k}_{g_{j'}}^{(2)}$$

$$\implies e_{j,j'}^{(1)} = e_{g_j,g_{j'}}^{(2)}$$

CAN SELF-ATTENTION MODEL WORD ORDER? I

- So, $\forall_j \vec{o}_j^{(1)} = \vec{o}_{g_j}^{(2)}$
- In other words: The representation of mary is identical to that of mary, and the representation of john is identical to that of john

CAN SELF-ATTENTION MODEL WORD ORDER? I

- **Question:** Can the other layers in the Transformer architecture (feed-forward net, layer normalization) help with the problem?
  - No, because they are apply the same function to all positions.
- **Question:** Would it help to apply more self-attention layers?
  - No. Since the representations of identical words are still identical in $\vec{O}$, the next self-attention layer will have the same problem.
- So... does that mean the Transformer is unusable?
- Luckily not. We just need to ensure that input embeddings of identical words at different positions are not identical.

POSITION EMBEDDINGS I

- Add to every input word embedding a position embedding $\vec{p} \in \mathbb{R}^d$:
- Input embedding of word "mary" in position j: $\vec{x}_j = \vec{w}_{\mathcal{I}(\text{mary})} + \mathbf{p}_j$

$$\vec{w}_{\mathcal{I}(\text{mary})} + \vec{p}_j \neq \vec{w}_{\mathcal{I}(\text{mary})} + \vec{p}_{j'} \text{ if } j \neq j'$$

- Option 1 ▸ Source: Vaswani et al., 2017
  (Deterministic) Sinusoidal position embeddings:

$$p_{j,i} = \begin{cases} \sin\left(\frac{j}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{j}{10000^{\frac{i-1}{d}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

- Option 2 ▸ Source: Devlin et al., 2019
  Trainable position embeddings: $\vec{P} \in \mathbb{R}^{J^{\max} \times d}$
  (Disadvantage: Cannot deal with sentences longer than $J^{\max}$)

PARALLELIZED ATTENTION I

- We want to apply our attention recipe to every query vector $\vec{q}_j$
- We could simply loop over all time steps $1 \leq j \leq J_x$ (or J_y) and calculate each $\vec{o}_j$ independently.
- Then stack all $\vec{o}_j$ into an output matrix $\vec{O} \in \mathbb{R}^{J_x \times d_v}$ (or $\mathbb{R}^{J_y \times d_v}$)
- But a loop does not use the GPU's capacity for parallelization, so it might be unnecessarily slow

PARALLELIZED SELF-ATTENTION I

- Do some inputs (e.g., $\vec{q}_j$) depend on previous outputs (e.g., $\vec{o}_{j-1}$)? If not, we can parallelize the loop into a single function:

$$\vec{O} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{X}; \theta)$$

- Attention in Transformers is usually parallelizable, unless we are doing autoregressive inference (more on that later).

PARALLELIZED SELF-ATTENTION I

- By the way: The Bahdanau model is not parallelizable in this way, because s_i (a.k.a. the query of the $i+1$'st step) depends on c_i (a.k.a. the attention output of the i'th step), see last lecture:

The hidden state s_i of the decoder given the annotations from the encoder is computed by

$$s_i = (1 - z_i) \circ s_{i-1} + z_i \circ \tilde{s}_i,$$

where

$$\begin{aligned}
\tilde{s}_i &= \tanh\left(WEy_{i-1} + U\left[r_i \circ s_{i-1}\right] + Cc_i\right) \\
z_i &= \sigma\left(W_z Ey_{i-1} + U_z s_{i-1} + C_z c_i\right) \\
r_i &= \sigma\left(W_r Ey_{i-1} + U_r s_{i-1} + C_r c_i\right)
\end{aligned}$$

PARALLELIZED SELF-ATTENTION I

- **Step 1**: The parallel application of the scaled dot product to all query-key pairs can be written as:

$$\vec{E} = \frac{\vec{Q}\vec{K}^T}{\sqrt{d_k}}; \quad \vec{E} \in \mathbb{R}^{J_x \times J_x}$$

$$\frac{1}{\sqrt{d_k}} \begin{bmatrix} - & \vec{q}_1 & - \\ & \vdots & \\ - & \vec{q}_{J_x} & - \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{k}_1 & \dots & \vec{k}_{J_x} \\ | & & | \end{bmatrix} = \begin{matrix} \\ \downarrow \\ queries \\ \downarrow \end{matrix} \begin{matrix} \rightarrow keys \rightarrow \\ \begin{bmatrix} e_{1,1} & \dots & e_{1,J_x} \\ \vdots & \ddots & \vdots \\ e_{J_x,1} & \dots & e_{J_x,J_x} \end{bmatrix} \end{matrix}$$

PARALLELIZED SCALED DOT PRODUCT SELF-ATTENTION I

- **Step 2**: Softmax with normalization over the second axis (key axis):

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J_x} \exp(e_{j,j''})}$$

- Let's call this new normalized matrix $\vec{A} \in (0,1)^{J_x \times J_x}$
- The rows of $\vec{A}$, denoted $\vec{\alpha}_j$, are probability distributions (one $\vec{\alpha}_j$ per $\vec{q}_j$)

PARALLELIZED SCALED DOT PRODUCT SELF-ATTENTION I

- **Step 3**: Weighted sum

$$\vec{O} = \vec{A}\vec{V}; \vec{O} \in \mathbb{R}^{J_x \times d_v}$$

$$\begin{bmatrix} - & \alpha_1 & - \\ & \vdots & \\ - & \alpha_{J_x} & - \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{v}_{:,1} & \dots & \vec{v}_{:,d_v} \\ | & & | \end{bmatrix} = \begin{matrix} \\ \downarrow \\ \textit{queries} \\ \downarrow \end{matrix} \begin{matrix} \overset{\rightarrow d_v \text{(value dims)} \rightarrow}{} \\ \\ \begin{bmatrix} o_{1,1} & \dots & o_{1,d_v} \\ \vdots & \ddots & \vdots \\ o_{J_x,1} & \dots & o_{J_x,d_v} \end{bmatrix} \end{matrix}$$

... AS A ONE-LINER I

$$\vec{O} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{X}; \theta) = \text{softmax}\Big(\frac{(\vec{X}\vec{W}^{(q)})(\vec{X}\vec{W}^{(k)})^T}{\sqrt{d_k}}\Big)(\vec{X}\vec{W}^{(v)})$$

- GPUs like matrix multiplications
  $\rightarrow$ usually a lot faster than RNN!
- But: The memory requirements of $\vec{E}$ and $\vec{A}$ are $\mathcal{O}(J_x^2)$
- Sequence lengths are an important issue.

ADD-ON: CHANGES FOR CROSS ATTENTION I

- **Step 1**: Scaled dot-product

$$\vec{E} = \frac{\vec{Q}\vec{K}^T}{\sqrt{d_k}}; \quad \vec{E} \in \mathbb{R}^{J_y \times J_x}$$

- **Step 2**: Softmax

$$\alpha_{j,j'} = \frac{\exp(e_{j,j'})}{\sum_{j''=1}^{J_x} \exp(e_{j,j''})}$$

- **Step 3**: Output

$$\vec{O} = \vec{A}\vec{V}; \vec{O} \in \mathbb{R}^{J_y \times d_v}$$

- As one-liner:

$$\vec{O} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{Y}; \theta) = \text{softmax}\left(\frac{(\vec{Y}\vec{W}^{(q)})(\vec{X}\vec{W}^{(k)})^T}{\sqrt{d_k}}\right)(\vec{X}\vec{W}^{(v)})$$

MULTI-LAYER ATTENTION I

- Sequential application of several attention layers, with separate parameters $\{\theta^{(1)} \ldots \theta^{(N)}\}$
- In Transformer: sequential application of Transformer blocks
- There are some additional position-wise layers inside the Transformer block, i.e., $\vec{O}^{(n)}$ undergoes some additional transformations before becoming the input to the next Transformer block $n + 1$

MULTI-HEAD ATTENTION I

- Application of several attention layers (“heads”) in parallel
- M sets of parameters $\{\theta^{(1)}, \ldots, \theta^{(M)}\}$, with $\theta^{(m)} = \{\vec{W}^{(m,q)}, \vec{W}^{(m,k)}, \vec{W}^{(m,v)}\}$
- For every head, compute in parallel:

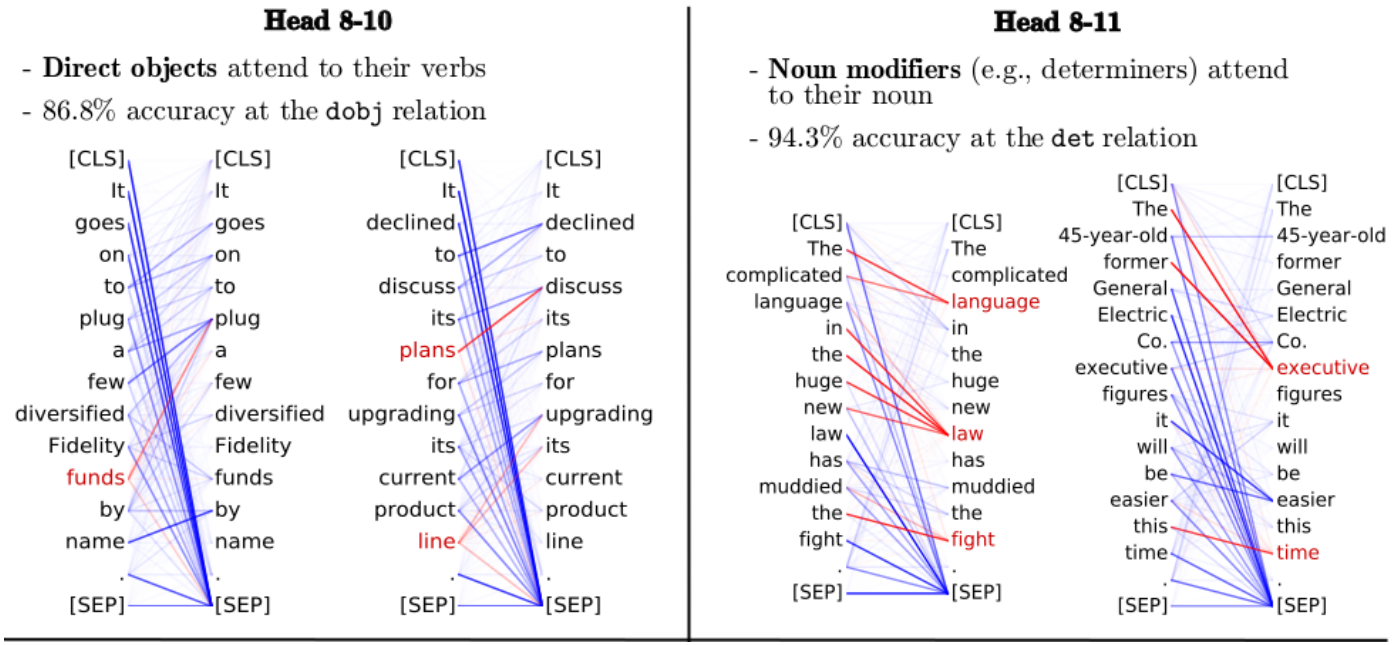
$$\vec{O}^{(m)} = \mathcal{F}^{\text{attn}}(\vec{X}, \vec{Y}; \theta^{(m)})$$

- Choice of dimensionality: $d_k = d_v = d_x/M$
- Concatenate all $\vec{O}^{(m)}$ along their last axis; then down-project the concatenation with an additional parameter matrix $\vec{W}^{(o)} \in \mathbb{R}^{Md_v \times d_x}$:

$$\vec{O} = [\vec{O}^{(1)}; \ldots; \vec{O}^{(M)}]\vec{W}^{(o)}$$

MULTI-HEAD ATTENTION I

- Conceptually, multi-head attention is to single-head attention like a filter bank is to a single filter (CNNs)
- Division of labor: different heads model different kinds of inter-word relationships



▸ Source: Clark et al., 2019

MODEL SIZE I

- Embedding size: $d_x = 512$
- Number of heads: $M = 8$
- Embedding size *per head*: $d_k = d_v = 512/8 = 64$

  $\rightarrow \vec{W}^{(q)} \in \mathbb{R}^{512\times64}$, $\vec{W}^{(k)} \in \mathbb{R}^{512\times64}$, $\vec{W}^{(v)} \in \mathbb{R}^{512\times64}$
- Projection matrix after multi-head attention: $\vec{W}^{(o)} \in \mathbb{R}^{512\times512}$